



Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	Julián Merino Sherman Abarca Matías Encarnación Matías Lavanda Jefferson Sarango
Ciclo	3ro A
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	02
Título de la Práctica	Implementación de acceso secuencial y aleatorio a archivos con métricas de I/O
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 8 de octubre 2026 - viernes 9 de octubre 2026.
Horario	jueves 8 de 08:30 – 09:30 viernes 9 de 11:30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar algoritmos de acceso secuencial para la persistencia masiva de datos en disco mediante el módulo nativo de I/O en JavaScript.

Cuantificar empíricamente la latencia computacional (tiempo de respuesta $T(n)$) y el impacto en la memoria Heap ($S(n)$) al procesar archivos de forma síncrona (bloqueante) versus flujos asíncronos (*Streams*).

Evaluar las decisiones de diseño físico en el acceso a almacenamiento mecánico o de estado sólido bajo los principios de *Green Computing*,



unl

Universidad
Nacional
de Loja

analizando cómo la optimización de los accesos I/O reduce el consumo energético del hardware.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial de Node.js (módulo fs).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).
- Recursos Digitales: Sílabo de la asignatura, repositorio base en GitHub.
- Plataforma EVA UNL.

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos (recomendado 4).
- Memoria RAM: Mínimo 4 GB.
- Almacenamiento: Disco HDD o SSD con al menos 1 GB de espacio libre temporal.

5. Procedimiento / Metodología.

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo satelital del APE 001 ahora necesita leer el millón de coordenadas ($N = 10^6$) que han sido transmitidas y almacenadas en un archivo físico en el servidor. El problema radica en que leer el archivo de una sola vez colapsa la memoria RAM del contenedor en la nube. Se debe diseñar una estrategia para leer los datos con el mínimo impacto temporal y espacial.
- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo interactúa el *Event Loop* de JavaScript (motor V8 y libuv) con las llamadas del sistema operativo (Syscalls) para el acceso a disco.
- **Modalidad:** Individual y análisis comparativo grupal posterior.

Tarea 1: Generación del Conjunto de Datos Masivo (Mock Data)

Para poder medir el I/O, primero necesitamos un archivo físico considerable.

1. En el directorio de su proyecto, cree un archivo llamado lab02_generador.js.

2. Implemente el siguiente código para escribir un archivo CSV con 10^6 registros utilizando escritura en flujo (Stream) para no saturar la memoria durante su creación.

```
const fs = require('fs');
const { performance } = require('perf_hooks');

const FILE_NAME = 'coordenadas_masivas.csv';
const N = 1000000;

console.log(`[Generador] Iniciando escritura de ${N} registros...`);
const start = performance.now();

// Utilizamos un Stream de escritura (Write Stream)
const writableStream = fs.createWriteStream(FILE_NAME);
writableStream.write('id,latitud,longitud\n'); // Cabecera

for (let i = 0; i < N; i++) {
  const lat = (Math.random() * 180 - 90).toFixed(6);
  const lng = (Math.random() * 360 - 180).toFixed(6);
  // Buffer string format
  writableStream.write(`${i},${lat},${lng}\n`);
}

writableStream.end();

writableStream.on('finish', () => {
  const end = performance.now();
  console.log(`[Generador] Archivo creado en ${((end - start) / 1000).toFixed(2)} segundos.`);
  // Calcular tamaño del archivo en disco
  const stats = fs.statSync(FILE_NAME);
  const sizeInMB = stats.size / (1024 * 1024);
  console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
});
```

3. Ejecute en terminal: `node lab02_generador.js` y verifique la creación del archivo `coordenadas_masivas.csv`.

Tarea 2: Enfoque 1 - Lectura Síncrona (Bloqueante / $O(n)$ Espacial)

Este enfoque representa la manera más intuitiva, pero menos eficiente, de procesar un archivo físico.

1. Cree un archivo llamado lab02_lectura_sync.js.
2. Implemente el código para cargar todo el archivo en la RAM.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura Síncrona] Iniciando carga en memoria...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
// I/O Bloqueante: El hilo principal se detiene hasta cargar todo
const data = fs.readFileSync(FILE_NAME, 'utf-8');
// Simulamos procesamiento separando por saltos de línea
const lineas = data.split('\n');
const end = performance.now();
const memoryAfter = process.memoryUsage().heapUsed;
console.log(`[Lectura Síncrona] Total registros leídos: ${lineas.length - 1}`);
console.log(`[Lectura Síncrona] Tiempo de I/O + Parsing: $(((end - start) / 1000).toFixed(2)) segundos.`);
console.log(`[Lectura Síncrona] Consumo Neto de RAM: $(((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)) MB`);
```

3. Ejecute node lab02_lectura_sync.js y tabule el tiempo y el impacto en memoria.

Tarea 3: Enfoque 2 - Lectura Asíncrona con Streams ($O(1)$ Espacial)

1. Cree el archivo lab02_lectura_stream.js.
2. Implemente la abstracción de flujos (Streams) para leer fragmentos (*chunks*) secuenciales, reduciendo la complejidad espacial teórica de $S(n) = n \cdot \text{bytes}$ a un constante $S(n) = O(1)$ correspondiente al tamaño del *buffer*.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
```



```
console.log(`[Lectura por Stream] Iniciando procesamiento...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
let totalRegistros = 0;
// Creamos un flujo de lectura secuencial no bloqueante
const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8' });
readableStream.on('data', (chunk) => {
  // Procesamos el chunk de datos actual sin almacenarlo completo
  // Contamos cuántos saltos de línea hay en este fragmento
  let lineBreakCount = (chunk.match(/\n/g) || []).length;
  totalRegistros += lineBreakCount;
});
readableStream.on('end', () => {
  const end = performance.now();
  const memoryAfter = process.memoryUsage().heapUsed;
  console.log(`[Lectura por Stream] Total registros procesados: ${totalRegistros}`);
  console.log(`[Lectura por Stream] Tiempo de I/O parcializado: $${((end - start) / 1000).toFixed(2)} segundos.`);
  console.log(`[Lectura por Stream] Consumo Neto de RAM: $${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
});
```

3. Ejecute node lab02_lectura_stream.js. Compare drásticamente la métrica del consumo neto de RAM entre la Tarea 2 y la Tarea 3.

Tarea 4: Análisis Matemático de Rendimiento (ABI)

Basado en los resultados empíricos, los estudiantes investigarán el diseño del bus de datos y el *Thread Pool* interno (libuv) que gestiona estas llamadas.

- Demuestre teóricamente (utilizando notación de orden asintótico) por qué la función bloqueante impone $S(n) = O(n)$ frente al enfoque *Stream* $S(n) = O(1)$.
- Estructure su respuesta relacionándola con el uso energético del servidor (menos RAM implica menor recolección de basura y menor gasto térmico del procesador).

6. Resultados esperados:

Link al repositorio con actividades en GitHub:

https://github.com/Jullxn28/Ciclo3_EstructuraDeDatos_Practica

- Resultados Tarea 1:

```
APE2_EstructuraDeDatos_Grupo6 > JS lab02_generador.js > ...
1  const fs = require('fs');
2  const { performance } = require('perf_hooks');
3
4  const FILE_NAME = 'coordenadas_masivas.csv';
5  const N = 1000000;
6
7  console.log(`[Generador] Iniciando escritura de ${N} registros...`);
8  const start = performance.now();
9
10 // Utilizamos un Stream de escritura (Write Stream)
11 const writableStream = fs.createWriteStream(FILE_NAME);
12 writableStream.write('id,latitud,longitud\n'); // Cabecera
13
14 for (let i = 0; i < N; i++) {
15   const lat = (Math.random() * 180 - 90).toFixed(6);
16   const lng = (Math.random() * 360 - 180).toFixed(6);
17   // Buffer string format
18   writableStream.write(`${i},${lat},${lng}\n`);
19 }
20
21 writableStream.end();
22
23 writableStream.on('finish', () => {
24   const end = performance.now();
25   console.log(`[Generador] Archivo creado en ${((end - start) / 1000).toFixed(2)} segundos.`);
26   // Calcular tamaño del archivo en disco
27   const stats = fs.statSync(FILE_NAME);
28   const sizeInMB = stats.size / (1024 * 1024);
29   console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
30 });
31
```

```
[Running] node
"c:\Users\julia\OneDrive\Documentos\TERCERCICLO_COMPUTACION\EstructuraDeDatos\UNIDAD1\APE\APE_JavaScript\APE2_EstructuraDeDatos_Grupo6\lab02_generador.js"
[Generador] Iniciando escritura de 1000000 registros...
[Generador] Archivo creado en 4.89 segundos.
[Generador] Peso físico en disco: 26.86 MB
[Done] exited with code=0 in 5.138 seconds
```

- Resultados Tarea 2:

```
APE2_EstructuraDeDatos_Grupo6 > JS lab02_lectura_sync.js > ...
1  const fs = require('fs');
2  const { performance } = require('perf_hooks');
3  const FILE_NAME = 'coordenadas_masivas.csv';
4  console.log(`[Lectura Sincrona] Iniciando carga en memoria...`);
5  const memoryBefore = process.memoryUsage().heapUsed;
6  const start = performance.now();
7  // I/O Bloqueante: El hilo principal se detiene hasta cargar todo
8  const data = fs.readFileSync(FILE_NAME, 'utf-8');
9  // Simulamos procesamiento separando por saltos de línea
10 const lineas = data.split('\n');
11 const end = performance.now();
12 const memoryAfter = process.memoryUsage().heapUsed;
13 console.log(`[Lectura Sincrona] Total registros leídos: ${lineas.length - 1}`);
14 console.log(`[Lectura Sincrona] Tiempo de I/O + Parsing: ${((end - start) / 1000).toFixed(2)} segundos.`);
15 console.log(`[Lectura Sincrona] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
16
```

```
PS C:\Users\julia\OneDrive\Documentos\TERCERCICLO_COMPUTACION\EstructuraDeDatos\UNIDAD1\APE\APE_JavaScript\APE2_EstructuraDeDatos_Grupo6> node lab02_lectura_sync.js
[Lectura Sincrona] Iniciando carga en memoria...
[Lectura Sincrona] Total registros leídos: 1000001
[Lectura Sincrona] Tiempo de I/O + Parsing: 0.23 segundos.
[Lectura Sincrona] Consumo Neto de RAM: 64.75 MB
PS C:\Users\julia\OneDrive\Documentos\TERCERCICLO_COMPUTACION\EstructuraDeDatos\UNIDAD1\APE\APE_JavaScript\APE2_EstructuraDeDatos_Grupo6>
```

- Resultados Tarea 3:

```

APE2_EstructuraDeDatos_Grupo6 > JS lab02_lectura_stream.js > ...
1  const fs = require('fs');
2  const { performance } = require('perf_hooks');
3  const FILE_NAME = 'coordenadas_masivas.csv';
4  console.log('[Lectura por Stream] Iniciando procesamiento...');
5  const memoryBefore = process.memoryUsage().heapUsed;
6  const start = performance.now();
7  let totalRegistros = 0;
8  // Creamos un flujo de lectura secuencial no bloqueante
9  const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8' });
10 readableStream.on('data', (chunk) => {
11     // Procesamos el chunk de datos actual sin almacenarlo completo
12     // Contamos cuántos saltos de línea hay en este fragmento
13     let lineBreakCount = (chunk.match(/\n/g) || []).length;
14     totalRegistros += lineBreakCount;
15 });
16 readableStream.on('end', () => {
17     const end = performance.now();
18     const memoryAfter = process.memoryUsage().heapUsed;
19     console.log('[Lectura por Stream] Total registros procesados: ${totalRegistros}');
20     console.log('[Lectura por Stream] Tiempo de I/O parcializado: ${((end - start) / 1000).toFixed(2)} segundos. ');
21     console.log('[Lectura por Stream] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB');
22 });
23

```

```

PS C:\Users\julia\OneDrive\Documentos\TERCERCICLO_COMPUTACION\EstructuraDeDatos\UNIDAD1\APE\APE_JavaScrip\APE2_EstructuraDeDatos_Grupo6> node lab02_lectura_stream.js
[Lectura por Stream] Iniciando procesamiento...
[Lectura por Stream] Total registros procesados: 1000001
[Lectura por Stream] Tiempo de I/O parcializado: 0.18 segundos.
[Lectura por Stream] Consumo Neto de RAM: 1.41 MB

```

- Resultados Tarea 4:

Comparación de función bloqueante frente al enfoque Stream

- Función bloqueante $S(n) = O(n)$: Al utilizar métodos sincrónicos (como `readFileSync`), el hilo principal y libuv cargan el 100% del archivo en la memoria de una sola vez. Si el tamaño del archivo es n , el espacio alojado en la RAM crece de forma lineal y directamente proporcional a n .
- Enfoque Stream $S(n) = O(1)$: La lectura por streams procesa los datos en pequeños fragmentos (chunks) a través de un buffer de capacidad limitada. Independientemente de si el archivo pesa (n) espacio de almacenamiento, el espacio de memoria requerido en cualquier instante siempre es constante, y se limita exclusivamente al tamaño fijo del buffer.

Impacto de rendimiento (RAM)

- El enfoque $O(n)$ satura rápidamente la RAM con archivos grandes. Esta carga masiva obliga al Garbage Collector a ejecutarse de manera brusca y continua para liberar el espacio utilizado.
- El enfoque $O(1)$ mantiene el uso de RAM al mínimo. Al exigir menos memoria, el Garbage Collector casi no interviene, liberando a la CPU de trabajo innecesario y manteniendo la eficiencia energética y evitando el sobre gasto térmico.

Los ciclos de CPU dedicados a esta intensa recolección de basura generan una sobrecarga de procesamiento, lo que se traduce directamente en un mayor gasto térmico y consumo eléctrico.

- Se obtendrán métricas directas que evidencian cómo una operación aparentemente trivial de lectura/escritura puede causar cuellos de botella de I/O (*Input/Output Bottleneck*). El uso de *Streams* demostrará que la persistencia masiva puede gestionarse secuencialmente manteniendo una huella de memoria plana, logrando un código ambiental y computacionalmente sostenible.

Comparación de Enfoques

- Enfoque Síncrono (O (n)): La función bloqueante consumió 64.75 MB de RAM y tardó 0.23 segundos. Al cargar la totalidad de los datos de golpe, el consumo de memoria en el servidor fue muy alta, superando incluso el peso físico del archivo original (26.86 MB).
- Enfoque Streams (O (1)): La lectura secuencial consumió apenas 1.41 MB de RAM y a la vez más rápida, tardando 0.18 segundos.

Análisis

- El uso de Streams demostró que se puede gestionar la lectura de un millón de registros (26.86 MB) utilizando una cantidad de memoria constante y mínima (1.41 MB).
- El enfoque por *Streams* redujo el consumo de RAM en más de un 75% comparado con el método síncrono (de 64.75 MB a 1.41 MB) lo que se traduce a mejor el tiempo de respuesta consumiendo recursos mínimos.
- Al mantener el consumo en 1.41 MB, la memoria RAM no se satura. Esto evita que el Garbage Collector se dispare constantemente para liberar los 64.75 MB de objetos cacheados que genera el enfoque síncrono. Al reducir los ciclos de CPU destinados a recolectar basura, se efectúa un menor gasto térmico y energético.

7. Preguntas de Control:

1. **Causa Material:** Al realizar una lectura síncrona mediante `readFileSync`, ¿cuál es la "materia" que se satura en el ordenador y cómo se diferencia físicamente del estado de reposo de un archivo almacenado magnética o electrónicamente en el disco duro?

La materia que se satura es la memoria RAM (el Heap de Node.js). A diferencia del disco duro, donde el archivo está en reposo pasivo mediante celdas NAND o magnetismo sin consumir energía directa, en la RAM los datos se cargan de golpe como impulsos eléctricos activos. Eso exige un refresco constante de corriente y llena de golpe el espacio de memoria del servidor (~64.75 MB).

2. **Causa Formal:** El archivo CSV (Valores Separados por Comas) posee una "forma" específica (texto secuencial lineal). ¿De qué manera esta forma impone restricciones algorítmicas sobre nuestra capacidad para acceder aleatoriamente a la coordenada $N = 500,000$ sin tener que recorrer las $499,999$ coordenadas previas?

La restricción la impone el formato del archivo: al ser un CSV, cada línea tiene un tamaño de bytes variable según los dígitos de las coordenadas. Como la computadora no puede adivinar dónde empieza exactamente la coordenada $N = 500,000$ mediante una fórmula matemática, no hay forma de saltar directo a esa posición; obligatoriamente tiene que escanear los bytes de los $499,999$ registros anteriores buscando los saltos de línea ($\backslash n$).

3. **Causa Eficiente:** Durante el uso de *Streams* asíncronos en Node.js, el *Event Loop* delega la operación al sistema operativo. En este contexto, ¿quién actúa como el verdadero motor (causa eficiente) que transfiere los electrones desde el disco hasta la memoria RAM en fragmentos (chunks)?

La causa eficiente no es una única entidad, sino una cadena jerárquica de agentes mecánicos, electrónicos y de software:

1. A nivel de Software / Runtime: El motor libuv (la biblioteca de E/S asíncrona subyacente en Node.js) y el Kernel del Sistema Operativo, los cuales gestionan la llamada al sistema (syscall como `read()`) mediante subprocesos del Thread Pool sin bloquear el hilo principal.
2. A nivel de Controlador / Firmware: El controlador Direct Memory Access (DMA) junto con la controladora del bus I/O.
3. A nivel Físico (El verdadero motor): El controlador DMA (Acceso Directo a Memoria) es la causa eficiente primaria del hardware. Libera a la CPU de la tarea manual de mover datos byte a byte, orquestando directamente las señales eléctricas y los pulsos de reloj a través del bus de datos del sistema para transferir los paquetes de electrones (chunks de buffer) desde las celdas del disco (HDD/SSD) hacia las direcciones físicas de la memoria RAM.

4. **Causa Final:** Si todo dato en ejecución debe residir en la memoria principal para ser computado por la CPU, ¿cuál es el propósito (telos) de la "persistencia" en el disco físico, sabiendo que las operaciones de I/O añaden un costo de tiempo monumental en la escala de microsegundos (μs) frente a los nanosegundos (ns) de la memoria caché?

El propósito de la persistencia es garantizar la permanencia de la información (la RAM es volátil y pierde todo al apagarse) y permitir la escalabilidad. Aunque el disco sea mil veces más lento que la caché o la RAM, nos da un espacio de almacenamiento gigante e indefinido. La RAM solo actúa como una ventana temporal para procesar los datos por partes mediante *Streams*.

5. **Esencia y Existencia:** Un conjunto de 10^6 coordenadas existe como un bloque monolítico en el archivo `data.csv`, pero en el enfoque de *Streams*, solo una porción existe en la RAM a la vez. ¿Es conceptualmente el "mismo" conjunto de

datos procesado, si en la memoria jamás llegó a existir de manera unificada? Argumente.

Sí, sigue siendo el mismo conjunto de datos. Su esencia está dada por el contenido y el orden de la información, lo cual no cambia por la forma en que se lee. Procesarlo por Streams es equivalente a ver una película: no necesitas proyectar todos los fotogramas al mismo tiempo en la pantalla para haber visto la película completa; basta con que exista una continuidad temporal en el procesamiento de cada fragmento.

8. Conclusiones

- El procesamiento masivo de datos mediante funciones síncronas o bloqueantes impone una complejidad espacial de tipo lineal $S(n) = O(n)$, ya que carga la totalidad del archivo de golpe en la memoria RAM (Heap de Node.js). En contraste, la utilización de flujos asíncronos (Streams) reduce esta complejidad a un nivel constante $S(n) = O(1)$. En el caso práctico evaluado, el enfoque por Streams redujo el consumo de RAM en más de un 75% (de 64.75 MB a 1.41 MB) para un archivo de un millón de registros
- Minimizar el impacto en la memoria RAM evita que el recolector de basura se active de manera constante y agresiva. Esto reduce de forma directa los ciclos de trabajo innecesarios de la CPU, lo que se traduce en una optimización energética, una disminución del gasto térmico del hardware y el cumplimiento de principios de desarrollo sostenible (*Green Computing*).
- Procesar la información fragmentada mediante flujos secuenciales no altera la esencia ni la integridad del conjunto de datos global, los Streams permiten computar volúmenes masivos de información manteniendo una huella de memoria plana y segura.

9. Declaración de IA

Durante el desarrollo del presente trabajo se utilizó ayuda de IA como Gemini, como ayuda para una correcta redacción del documento, además; se utilizó ChatGPT para la ayuda de síntesis de los códigos y sus posibles resultados, siendo así una ayuda para poder manejar correctamente el uso y la interpretación del entorno programable utilizado.

Prompt Gemini:

Revisa el siguiente documento y dame una lluvia de ideas para mejorar su redacción y pueda entenderse de mejor manera.

Prompt ChatGPT:

Tengo los siguientes códigos para completar las siguientes tareas, podrías decirme si están bien estructurados y como los puedo mejorar para completar la tarea de mejor manera.

10. Bibliografía

[1] P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: <https://open.palni.org/>



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

[2] Node.js Foundation, "File system | Node.js v20.x Documentation", *Node.js API*, 2023. [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/fs.html>

[3] S. Tilkov et al., *Node.js in Action*, 2nd ed. Manning Publications, 2017.